Midterm Report: An IoT System for Real-time Human Activity Recognition

Tianchen Liu*, Juntao Hu*

***Abstract*—This report records the midterm state of the smartphone–laptop activity system proposed for IERG5230. A 4-bit SmolVLM-256M in the phone browser writes a short action phrase and a minimum token probability. A frame close to the previous one keeps the ongoing phrase and skips that model. The picture stays on the phone when the new phrase is confident and still matches the ongoing action. A confident mismatch must occur on two frames before the buffered frames are uploaded. Low token confidence uploads on the current frame. Qwen3.5-2B on the laptop then writes its own phrase. The phrases are compared by text-encoder cosine similarity, not by a closed caption bank. On 60 still images the cosine matcher scores 38/60 for SmolVLM and 40/60 for Qwen. Replaying the live rule on three frames per clip scores 1640/3783 on UCF101 and 660/1530 on HMDB51, below a single Qwen pass over the same frames. The latency predictor, the dynamic uplink budget, and the multi-client scheduler are specified and are not running yet.**

I. Introduction

An interactive system has to name the activity while the person is still moving. A closed label such as "sitting" is too coarse for that response. Sitting still, sitting and looking at a phone, and sitting and waving call for different responses, yet a small label set collapses them into one class. The system therefore describes the activity with a short English phrase. A strong vision-language model can write that phrase, but running it on every captured frame is too expensive for a phone, and uploading every frame makes the user wait on the uplink.

The design splits the work. SmolVLM-256M-Instruct runs in the phone browser and writes the phrase before any picture leaves the device [1]. The confidence of the phrase is the smallest probability among the newly generated tokens. The JPEG stays on the phone when this confidence is high and the phrase still agrees with the action already on screen. Otherwise the phone uploads the last few frames. A laptop on the same Wi-Fi network runs Qwen3.5-2B, writes its own phrase, and the screen keeps or replaces the ongoing phrase according to the cosine similarity of the two texts.

Two choices in the proposal have been changed because they fought that goal. The proposal kept a fixed sentence bank and let CLIP retrieve one bank sentence for the phone text and another for the picture. The screen then showed the bank sentence. The running system keeps the model's own phrase. The bank is used only when a free phrase must be scored against the 15 classes of the still-image evaluation. The proposal also planned a compute-aware upload scheduler. That scheduler is the networking half of the project. This midterm finishes the sentence-and-check path, including a causal check over the last few frames, and states the scheduler work that is still ahead.

The sentence path now has two extra gates in front of that upload. A live frame is reduced to a 32×32 grayscale image. When an action is already on screen, the leave count is zero, and the mean absolute difference from the previous frame is below 10, the phone keeps the phrase and does not run SmolVLM. A confident phrase that leaves the action is held for one frame. The buffered frames go to Qwen only on a second mismatch. A missing or low token probability does not wait. On a 60-image sample the text-encoder matcher scores 38/60 for the phone model and 40/60 for Qwen. The same live rule, replayed on three frames from each test clip, scores 1640/3783 on UCF101 and 660/1530 on HMDB51. A single Qwen pass over those three frames scores 1785/3783 and 812/1530.

The rest of this report is organized as follows. Section II places the prototype among activity recognizers, vision-language models, and edge offloading systems. Section III states the objectives and which of them this midterm meets. Section IV gives the keep rule, the three-frame check, and the upload control that is still to be built. Section V describes the browser client and the laptop server. Section VI reports the still-image measurement and the three-frame replay on UCF101 and HMDB51. Section VII states what the measurement supports and what remains before the presentation on 20 November 2026.

II. Literature Study

A. Activity labels and free phrases

Activity recognizers take RGB, skeleton, IMU, radar, or Wi-Fi, and usually return one label from a closed set. An open-set model can reject an unknown action. It still does not describe that action. An open-vocabulary model can match a clip to a class name while the action is still going on. Free-form captions also exist. OV-HAR turns a sensor window into a sentence [2], and OpenHOUSE calls a vision-language model for a free-form description of streaming video [3]. Both are evaluated on recorded data, not on a live phone link.

Recent video benchmarks such as Kinetics-400 are clips of about ten seconds. A single decoded frame, or three frames a second apart, does not support a claim on that benchmark. Online action detection reads only past frames and emits a label at every time step. The prototype is causal, in that it never reads a frame that has not been captured, but it updates only as fast as the phone model finishes one still image. The missing piece in the cited caption systems is a sentence produced on the phone, before any picture is uploaded, while the person is still moving.

* equal contribution.

Progress report, 28 October 2026. The final presentation is on 20 November 2026.

B. Retrieval and generative vision-language models

CLIP retrieves a sentence from a fixed bank by cosine similarity between an image embedding, or a text embedding, and the bank [4]. It does not write a new sentence. That retrieval was the checker in the proposal: the phone phrase and the uploaded picture were each mapped to one bank sentence, and the screen showed the bank sentence. A generative vision-language model writes the sentence token by token, and accuracy follows model size. Qwen2-VL scales this family up to 72B parameters [5]. Models that can sit on a phone also exist. MiniCPM-V is in the 2B–8B range [6], and SmolVLM-256M fits in under 1 GB [1]. Published scores for these models cover documents, charts, and general questions, not fine differences between similar activities.

A small generative model can therefore write the sentence on the phone. It does not, by itself, say which of those sentences are safe to leave on the screen. The checker in this report follows the text half of the CLIP comparison and not the image encoder. Both the phone phrase and the laptop phrase are encoded by a text embedding model, and the score is their cosine [7]. Mapping either phrase onto a class sentence for the still-image evaluation uses the same score:

$$c = \arg\max_{c \in \mathcal{C}} \cos\big(f_T(s), e_c\big). \tag{1}$$

The live screen does not apply (1). It shows s itself.

C. Client–server collaboration

Neurosurgeon profiles network layers and chooses where to split them, so the server receives image features and the user waits for that transfer [8]. Speculative decoding splits the work by tokens: a small model drafts, a large model verifies, and the server still reads the picture [9], [10]. Later edge variants keep the same split [11], [12]. edgeVLM answers on the device at once and reuses a late cloud caption as context for later frames, rather than as the result of the frame that is on screen now [13]. EdgeBoost uploads an input only when the local confidence is low. On image classification this removes about 20–55% of the communication of a cloud-only run, and the cloud still returns a class label [14].

What these systems return is a class, a feature tensor, or a caption that arrives after the frame has moved on. The prototype shows an activity phrase as soon as the phone finishes, and it sends the picture only when that phrase is unsafe relative to the action already on screen.

D. Communication after the phrase exists

A proportional-fair scheduler and an L4S queue assign service to packets that have already been formed. They do not look at how long the laptop will spend on the picture after it arrives [15], [16]. Semantic communication scores a transmission by task success rather than by throughput, but the payload is still a feature tensor [17]. Offloading chooses local or remote execution, and adaptive streaming changes how many bytes are sent. After the sentence has already been written, these mechanisms still leave open whether the picture should follow, how large that picture should be, and which phone should be checked first.

TABLE I
OBJECTIVES AND MIDTERM STATUS

Objective	Status
Write an English action phrase on the phone before any picture is uploaded	Done
Keep the picture on the phone when the phrase is confident and still matches the ongoing action	Done
Let the laptop write its own phrase and compare the two phrases by text cosine	Done
Check a short causal clip of the last three frames when the local test fails	Done
Score free phrases on 60 still images and on three-frame UCF101 and HMDB51 clips	Done
Predict checker latency, set a per-frame uplink budget, and schedule several phones	Planned

The proposal takes that last decision. Token confidence decides whether the JPEG is uploaded. JPEG quality and resolution decide its size. For every upload, the predicted edge latency sets the uplink deadline, and an application-layer scheduler serves the earliest deadline first. The running prototype implements the confidence decision and a fixed JPEG size. The predictor and the scheduler are specified in Section IV and are not in the code yet.

III. OBJECTIVES

The project has an information-processing half and a networking half. The information-processing half is the phrase on the phone and the check on the laptop. The networking half is the decision of when that check is allowed to occupy the uplink. Table I lists the objectives and the midterm status of each one.

The first objective is a phrase, not a class identifier. The phone model must finish generation, and the page must show the phrase, without waiting for the laptop. The confidence attached to that phrase is the minimum probability of the new tokens, so a single uncertain token can force a check.

The second objective is the upload gate. A high confidence is not enough on its own once an action is already on screen. The new phrase must also stay close to that action. The picture is the payload that this gate can suppress. The phrase itself may still be sent, because the text encoder runs on the laptop.

The third objective replaces the proposal's bank caption. The laptop model writes a free phrase. Agreement is the cosine of the two phrase embeddings. The screen keeps the model's words. A class sentence is introduced only in the evaluation, where a free phrase has to be counted against a label.

The fourth objective keeps the check causal and short. The phone reads the newest frame. The laptop, and only the laptop, may read the last three frames, captured about one second apart and uploaded in time order. The system does not wait for a future frame, and it does not claim a ten-second clip.

The fifth objective is a measurement that can be repeated. The models are not trained on either sample. The still-image check scores one free phrase against 15 class sentences. The video check replays the live rule on three frames from each clip in the UCF101 recognition test list and the HMDB51 test

list. Those frames are taken around the middle of the clip. The replay therefore scores the gate, the quiet skip, and the phrase, and it does not score the start or the end of an action.

The sixth objective is unchanged from the proposal and is not met at this midterm. Before an upload, the laptop should predict how long the checker will take, subtract that time from the confirmation deadline, and turn the remainder into an uplink budget. When several phones ask for a check at once, the earliest uplink deadline should go first. The November work is to collect the traces, train the predictor, and compare that rule with round-robin and with a static budget.

IV. Design

Figure 1 is the path that is running. A quiet frame keeps the phrase and skips the phone model. Otherwise the phone writes and shows a phrase before any picture leaves the device. The sentence crosses the Wi-Fi link for a semantic check. The JPEG follows immediately when token confidence is low, and only on a second frame when a confident phrase has left the ongoing action. The latency predictor and the upload scheduler are not on this figure.

A. Local phrase and confidence

SmolVLM-256M-Instruct runs in the phone browser at 4-bit. For a frame x_i it writes a phrase s_i and a probability $p_{i,t}$ for each new token. The confidence is the weakest of those probabilities,

$$\kappa_i = \min_t p_{i,t}. \tag{2}$$

The phrase is shown when local generation finishes. It does not wait for the laptop. An empty phrase has confidence missing and is treated as unsafe. The phone prompt asks for a short action phrase and nothing else, so the later cosine compares two phrases of the same kind.

B. When the picture stays on the phone

Let a be the phrase of the action that is already on screen, and let τ_{u} and τ_{s} be the confidence and cosine thresholds. The JPEG stays on the phone when the local phrase is confident and still close to that action:

$$\kappa_i \geq \tau_{\mathrm{u}} \quad \text{and} \quad \cos\big(f_T(s_i), f_T(a)\big) \geq \tau_{\mathrm{s}}. \tag{3}$$

The defaults in the prototype are $\tau_{\mathrm{u}} = 0.5$ and $\tau_{\mathrm{s}} = 0.70$. The text encoder f_T is `bge-small-en-v1.5`, running on the laptop CPU so the GPU remains available for Qwen [7]. An empty phrase scores 0. The first confident phrase of a session starts the action. There is no a yet, so only the confidence test applies, and the picture is not uploaded.

The cosine in (3) is computed on the laptop. A confident live frame therefore still sends the phrase. It does not send the JPEG. One failure of that cosine test increments a leave count and keeps a. The buffered frames are uploaded only when the count reaches 2. The count returns to 0 when the phrases match, when Qwen returns a decision, and at the start of a new action. One-shot capture and a file upload do not use the action state, the leave count, or the quiet-frame test. They compare s_i with the laptop phrase only.

C. Quiet frames

Before SmolVLM runs, the live page scales the current JPEG to a 32×32 grayscale image g_i. The distance from the previous live frame is the mean absolute difference

$$d_i = \frac{1}{1024} \sum_{u=1}^{32} \sum_{v=1}^{32} \left| g_i(u,v) - g_{i-1}(u,v) \right|. \tag{4}$$

SmolVLM is skipped, and a stays on screen, when an action is already set, the leave count is 0, and $d_i < 10$. A frame that is waiting for a second mismatch is not treated as quiet, so a pending leave is not erased by a small pixel change. The first frame of a session has no previous image and always reaches the model. The same threshold and the same 32×32 signature are used by the video replay.

D. Three-frame check

Live mode stores one JPEG per second and keeps the last three. SmolVLM is invoked on the newest frame only, and only when (4) does not skip it. While it runs, newer captures replace the buffer and are not queued behind the frame already in inference. The upload uses those frames in time order. It happens on the current frame when κ_i is missing or below τ_{u}. It happens on a confident frame only after two cosine failures. Qwen3.5-2B then writes one phrase q_i for the frames it receives. The prompt states that the frames are about one second apart. A single uploaded frame uses the same short-action prompt as the phone.

The ongoing phrase a is kept when q_i is close to it. Otherwise the screen uses s_i when q_i is close to s_i or when q_i is empty, and uses q_i when the two phrases disagree:

$$a^+ = \begin{cases} a & \text{if } a \neq \emptyset \text{ and } \cos(f_T(q_i), f_T(a)) \geq \tau_{\mathrm{s}}, \\ s_i & \text{if } q_i = \emptyset \text{ or } \cos(f_T(s_i), f_T(q_i)) \geq \tau_{\mathrm{s}}, \\ q_i & \text{otherwise.} \end{cases} \tag{5}$$

A kept action does not reset its start time. A replaced action starts at the newest frame. Algorithm 1 is the live procedure.

The phone model still reads one still image. Three frames are joint input only for Qwen, and only after the local check fails. The vision encoder of SmolVLM resizes its input and emits a fixed image-token sequence, so stitching the three frames into one JPEG would shrink each frame instead of adding time. The update period is the phone generation time. On the development phone with WebGPU that time is under three seconds, so actions shorter than the period can fall between updates. This is not per-frame online detection and not a Kinetics evaluation.

E. Upload control still to be built

The sentence that appears first is local, $T_i^{\mathrm{draft}} = T_i^{\mathrm{local}}$, where T_i^{local} is the SmolVLM time on the phone. A frame that fails (3) also waits for a checked phrase. Its confirmation delay is

$$T_i^{\mathrm{conf}} = T_i^{\mathrm{wait}} + T_i^{\mathrm{up}} + T_i^{\mathrm{edge}} + T_i^{\mathrm{down}}, \tag{6}$$

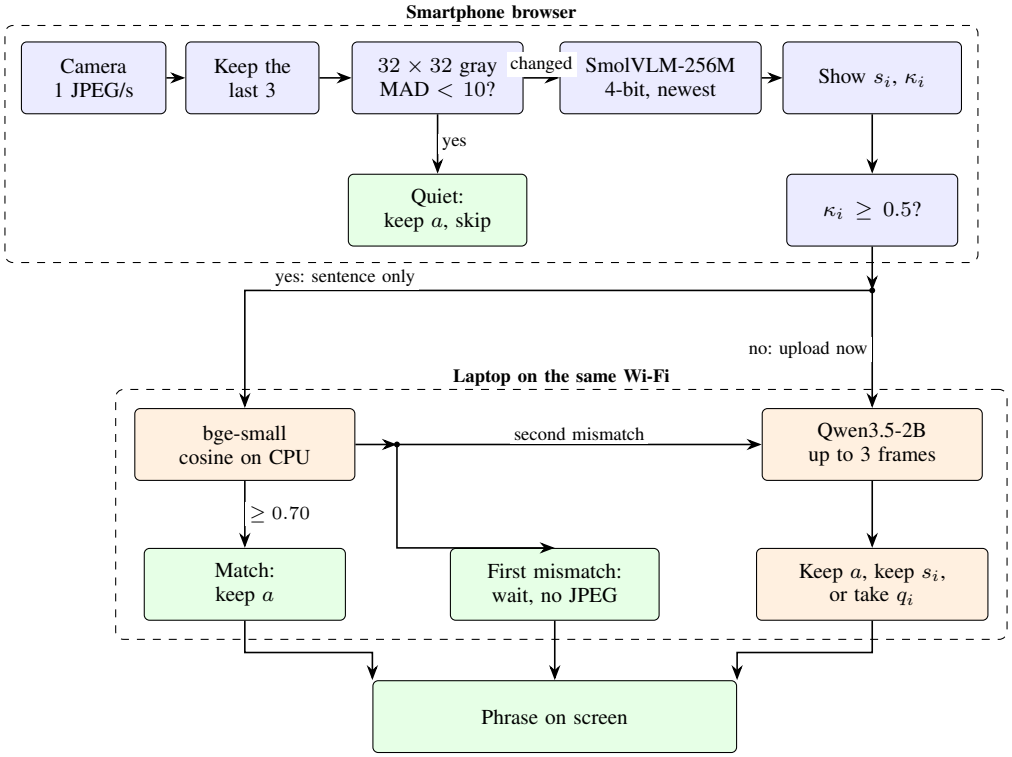


Fig. 1. Live framework. A quiet frame, measured by a 32×32 grayscale mean absolute difference below 10, keeps the ongoing phrase and skips SmolVLM. Otherwise the phone shows s_i at once. Low token confidence uploads the buffered frames immediately. A confident phrase sends the sentence only. The text encoder keeps the action when the sentence is still close. The first mismatch waits. The second mismatch uploads, and Qwen's phrase is kept only when it agrees with neither a nor s_i. The first confident phrase of a session starts a with no cosine test.

where T_i^{wait} is the time until an upload grant, T_i^{up} is the JPEG upload, T_i^{edge} is the checker, and T_i^{down} is the return of a short result. The request meets its deadline when $T_i^{\text{conf}} \leq D_i^{\text{conf}}$.

Checker time is not a constant. The predictor planned for November reads

$$\mathbf{z}_i = \left[L_i,\ U_i^{\text{CPU}},\ T_{i-1}^{\text{edge}} \right], \tag{7}$$

with L_i the JPEG size in bytes, U_i^{CPU} the server load at the request, and T_{i-1}^{edge} the previous checking time. A small network g_θ predicts $\hat{T}_i^{\text{edge}} = g_\theta(\mathbf{z}_i)$ before the JPEG is uploaded. The remaining uplink budget is

$$B_i^{\text{up}} = \left[D_i^{\text{conf}} - \hat{T}_i^{\text{edge}} - \hat{T}_i^{\text{down}} - M \right]^+, \tag{8}$$

and the uplink deadline is $d_i^{\text{up}} = a_i + B_i^{\text{up}}$ from the arrival time a_i. The scheduler serves

$$i^\star = \arg\min_{i \in \mathcal{Q}} d_i^{\text{up}}. \tag{9}$$

A long predicted edge time leaves less room for the upload, so that request gets an earlier deadline. None of g_θ, B_i^{up}, or the queue $\mathcal{Q}$ is in the prototype. The phone does not send a planned JPEG size or a confirmation deadline, and the laptop does not order uploads.

V. Implementation

A. Phone browser

The client is a smartphone browser. The page is served by the laptop. SmolVLM-256M-Instruct runs in a Web Worker through Transformers.js 3.7.1. The vision encoder and the decoder use 4-bit weights. Token embeddings stay in 16-bit. WebGPU is selected when the browser exposes an adapter. If that adapter supports `shader-f16`, the 4-bit weights are stored in the 16-bit WebGPU variant. Otherwise the worker uses the plain 4-bit variant. If WebGPU fails to start the model, the worker loads the same 4-bit weights on WASM. WASM uses up to four threads when the page is cross-origin isolated and the device is not an iPhone. The laptop sends `Cross-Origin-Opener-Policy` and `Cross-Origin-Embedder-Policy` so a desktop browser can take that threaded fallback. On the development phone, WebGPU brings one-frame generation under three seconds.

The generation call in the browser does not return a score tensor. The worker therefore installs a logits processor, reads the logits of each new token, and turns the maximum logit into a probability with a log-sum-exp. The confidence reported to the page is the minimum of those probabilities, which is κ_i in (2). Generation is greedy and stops at 16 new tokens. The prompt is "What is the person doing? Answer with only a short action phrase." The vision encoder resizes its input and emits a fixed image-token sequence, so the worker does not split the image into tiles, and it scales the longest side to 512 before inference. A smaller JPEG would be scaled back up inside the encoder.

The camera path stores one JPEG per second at quality 0.72, longest side 512, and keeps three blobs. A front camera

Algorithm 1 Live phrase with a quiet skip and a two-frame leave

Keep the last three JPEGs, one captured per second.
Compute d_i in (4). Let ℓ be the leave count.
if a is set and $\ell = 0$ and $d_i < 10$ **then**
  Keep a and skip SmolVLM.
else
  Run 4-bit SmolVLM on the newest frame and display s_i.
  $\kappa_i \leftarrow \min_t p_{i,t}$.
  if $\kappa_i < \tau_{\mathrm{u}}$ or κ_i is missing **then**
    Upload the buffered frames, oldest first.
  else if a is unset **then**
    Start a from s_i. Do not upload.
  else if $\cos(f_T(s_i), f_T(a)) \geq \tau_{\mathrm{s}}$ **then**
    Keep a and set $\ell \leftarrow 0$.
  else
    $\ell \leftarrow \ell + 1$.
    if $\ell < 2$ **then**
      Keep a. Do not upload.
    else
      Upload the buffered frames, oldest first.
    end if
  end if
end if
if frames were uploaded **then**
  Qwen writes q_i and apply (5).
  Set $\ell \leftarrow 0$.
end if

preview is mirrored. The blob that enters the model is mirrored back so the encoder sees an unmirrored frame. While a phrase is being generated, the capture clock continues and the buffer keeps only the newest three frames. The next inference starts from the newest frame after the current call returns. Before that call, the page draws the frame onto a 32×32 canvas, converts it to grayscale, and compares it with the previous live frame by (4). The page reads the threshold 10 and the leave limit 2 from the server health check, which returns the constants in the shared policy module. One-shot capture and a file chosen by the user send at most one image and do not update the ongoing action.

B. Laptop checker

The laptop serves the page with FastAPI and loads Qwen3.5-2B in bfloat16 on an RTX 5070 Laptop GPU. The phrase encoder, `bge-small-en-v1.5`, is loaded on the CPU. Mean pooling over the masked last hidden state, followed by ℓ_2 normalization, produces the vectors whose cosine is τ_{s}. One still image takes a few hundred milliseconds after the weights are resident. A spot check with two frames took 1.4 seconds. Qwen generation is also greedy, with at most 32 new tokens. Thinking mode is turned off. The image processor is capped at $28 \times 28 \times 256$ pixels, and each uploaded frame is again limited to a longest side of 512. A lock admits one Qwen call at a time.

The check endpoint receives the phone phrase, the ongoing phrase, and zero to three JPEGs. With no JPEG it only compares the two phrases. A cosine of at least 0.70 keeps the ongoing phrase and tells the phone not to upload. With one or more JPEGs the server runs Qwen. The single-frame prompt matches the phone. The multi-frame prompt adds that the images are in time order and about one second apart. The reply is one of four outcomes. The ongoing phrase is held when Qwen stays close to it. The phone phrase is confirmed when Qwen stays close to the phone and not to the ongoing action. The phone phrase is kept when Qwen returns an empty string. Otherwise the screen takes Qwen's phrase. The certificate is reissued for the current LAN address at every server start, so the phone can open the camera over HTTPS. This path does not change the Wi-Fi MAC or the access point.

C. Evaluation harness and division of work

The still-image harness loads the labeled train split of the Bingsu human action set, draws a stratified sample, and writes one phrase per image with the same short-action prompt. Cached phrases can be matched again without regenerating them. The class sentences are fixed English lines of the form "a person is running". The predicted class is (1). The video harness takes three frames from each clip, caches a SmolVLM phrase per frame and a Qwen phrase for each prefix length, and replays the quiet skip, the leave count, and the fusion rule in the same policy module the server uses. The phone worker is not part of either harness. Both load SmolVLM in float32 on CUDA, so those SmolVLM times are not the 4-bit browser times.

Both members integrate the demo, the evaluation, and the report. Juntao Hu is responsible for the local model, the acceptance rule, the laptop checker, and the caption-quality evaluation. That path is running. Tianchen Liu is responsible for the latency predictor, the dynamic uplink budget, the upload scheduler, and the multi-client simulation. That path has not been implemented.

VI. Evaluation

A. Protocol

The still-image check uses the Human Action Recognition set released by Bingsu. The published test split has placeholder labels, so the sample is drawn from the labeled train split: 15 classes, 4 images per class, seed 0, 60 images. The models are not trained on it. Each model sees one image and writes a free phrase with the same short-action prompt. The predicted class is the class sentence with the highest text-encoder cosine. An earlier matcher based on character overlap, with a bonus when the class name appeared in the phrase, is kept only as a reference.

This protocol scores (1). It does not score (3), (4), or (5). There is no ongoing action across the 60 images, and Qwen sees one frame rather than three. A phrase can describe the picture in ordinary English and still miss the class, because the class sentence is a single canonical line. "Typing" sits near both texting and using a laptop. The cosine can prefer either neighbor.

TABLE II
PHRASE-TO-CLASS ACCURACY ON 60 STILL IMAGES.

Matcher	SmolVLM	Qwen
Character overlap	27/60	30/60
Text-encoder cosine	38/60	40/60

TABLE III
COSINE MATCHES PER CLASS, OUT OF FOUR IMAGES.

Class	SmolVLM	Qwen	Mixed at 0.5
calling	4	1	4
clapping	2	3	3
cycling	3	4	4
dancing	2	1	2
drinking	3	4	3
eating	4	3	4
fighting	2	3	3
hugging	2	4	3
laughing	3	3	2
listening to music	2	2	2
running	4	4	4
sitting	2	2	2
sleeping	3	4	3
texting	1	1	1
using a laptop	1	1	1
Total	38	40	41

B. Phrase-to-class accuracy

Table II shows that the text encoder recovers paraphrases that character overlap misses. “Riding bike” matches cycling, and “Drinking wine” matches drinking, without the class name having to appear as a substring. The gain is 11 images for SmolVLM and 10 images for Qwen. Qwen is ahead of SmolVLM by two images under the cosine matcher. On this sample the larger model is not a large gain once both models are allowed to speak in their own words and a text encoder absorbs the wording.

Table III breaks the cosine score down by class. Both models are correct on all four running images and fail most texting and laptop images. SmolVLM is correct on all four calling images. Qwen writes “talking” or “using a phone” on several of those images, and the cosine then prefers texting or calling inconsistently, so Qwen scores 1/4 on calling. Qwen is correct on all four hugging, cycling, drinking, and sleeping images, where SmolVLM drops two or one. The errors cluster on actions that share a posture or an object: texting, using a laptop, and sitting; smiling against laughing; talking against calling. Several misses are confident. SmolVLM assigns 0.90 to “Taking picture” on a texting image and 0.84 to “Reading” on a sleeping image. A high minimum token probability means the generator was sure of its tokens. It does not mean the phrase is the class sentence.

C. Confidence as a router

Deferring to Qwen when the SmolVLM confidence is below a threshold uses the cosine predictions. Table IV reports three thresholds. At 0.5, the mixed result is 41/60 and 26 of the 60 images take the Qwen phrase. At 0.3 the result is 42/60 with 12 images deferred. At 0.7 it is 43/60 with 41 images deferred.

TABLE IV
DEFER TO QWEN WHEN SMOLVLM CONFIDENCE IS BELOW THE THRESHOLD.

Threshold	Correct	Images sent to Qwen
0.3	42/60	12
0.5	41/60	26
0.7	43/60	41

TABLE V
OFFLINE CUDA GENERATION TIME PER STILL IMAGE, IN MILLISECONDS.

Model	Min	Median	Mean	Max
SmolVLM, float32	468	538	582	1040
Qwen3.5-2B, bfloat16	184	210	235	1184

The gain over Qwen alone is at most three images. Raising the threshold uploads more and barely moves the class score.

The confidence values explain the flat curve. On the CUDA run, SmolVLM’s mean confidence is 0.577 on the 38 correct phrases and 0.484 on the 22 misses. At 0.5 the high-confidence slice is 22/34 correct and the low-confidence slice is 16/26 correct. The two rates are close, so the gate does not separate the phrases that need a second model from the phrases that are already right. Qwen’s own confidence separates a little more, 25/34 above 0.5 against 15/26 below, but the router never reads Qwen’s confidence. It has to decide before the upload.

The two models are complementary on the class labels, and the gate does not harvest that complementarity. Both phrases match the label on 30 images. SmolVLM alone is right on 8, Qwen alone is right on 10, and both are wrong on 12. An oracle that kept the correct phrase whenever either model produced one would score 48/60. The best threshold in Table IV scores 43/60. Laughing is a concrete loss: each model alone scores 3/4, and the mix at 0.5 scores 2/4, because the gate hands Qwen an image that SmolVLM already had right. The live rule in (5) can also refuse a Qwen phrase that drifts away from the ongoing action. The 60-image mix never applies that second test.

D. Offline generation time

Table V is the wall time of `eval_har.py` on CUDA after the weights are loaded. SmolVLM is float32 in that harness. Qwen is bfloat16, which is the server configuration. These numbers are not the phone. The phone runs 4-bit SmolVLM in the browser, and on the development phone that generation is under three seconds. The laptop figures say only that, once the picture has arrived, one-image Qwen generation is usually about 0.2 s, with one outlier at 1.2 s. A two-frame spot check outside this table took 1.4 s. The confirmation delay in (6) is therefore dominated by the phone generation period, the uplink, and any queue, not by a multi-second laptop decode of one JPEG. That split is why the November predictor has to be trained on traces that vary JPEG size and laptop load, rather than on a single constant.

TABLE VI
THREE-FRAME REPLAY. UPLOADS ARE QWEN CALLS PER CLIP. SKIPPED FRAMES BYPASS SMOLVLM.

Set	Branch	Correct	Mean ms	Uploads	Skipped
UCF101	SmolVLM	928/3783	350	0	29.5%
UCF101	Qwen	1785/3783	348	1.00	0
UCF101	Live rule	1640/3783	816	1.49	29.3%
HMDB51	SmolVLM	288/1530	373	0	24.3%
HMDB51	Qwen	812/1530	371	1.00	0
HMDB51	Live rule	660/1530	933	1.71	24.0%

E. Three-frame replay

The live rule is replayed on the UCF101 recognition test list and on the HMDB51 test list packaged with the Sina272 release. UCF101 contributes 3783 clips and 101 classes. HMDB51 contributes 1530 clips, 51 classes, and 30 clips per class. Each clip is reduced to three JPEGs, longest side 512. When the clip lasts at least 2.2 seconds the frames sit one second before the middle, at the middle, and one second after. A shorter clip spreads the three indexes across its length. The replay then walks those frames in order with Algorithm 1. SmolVLM-only uses the quiet skip and never uploads. Qwen-only sees all three frames in one call. The hybrid branch is the live rule, with $\tau_{\mathrm{u}} = 0.5$ and $\tau_{\mathrm{s}} = 0.70$.

The predicted class is again (1). The class sentence is the template "a person is" followed by the class name, with camel case split into words, so the sentence is not always grammatical. The models are not trained on either set. Latency sums the pixel check, the SmolVLM calls, the Qwen calls, and the phrase-encoder calls used on that clip. It does not include Wi-Fi time. SmolVLM is float32 on CUDA. Qwen is bfloat16. These times are not the phone. The three frames are taken around the middle of a finished clip, so the replay does not measure an action's start or end, and it is not the official accuracy of either dataset.

Table VI puts Qwen ahead of the live rule on both lists, by 145 clips on UCF101 and by 152 clips on HMDB51. SmolVLM alone is far behind both. The quiet skip removes 29.5% of UCF101 frames and 24.3% of HMDB51 frames from the phone model, and the live rule keeps almost the same skip rate. That saving does not make the live rule cheaper than one Qwen call. A clip can upload more than once, because an early low confidence sends the prefix seen so far and a later frame can send again. The mean is 1.49 uploads on UCF101 and 1.71 on HMDB51, against one upload for the Qwen branch. The 95th percentile compute is 1.6 s and 1.7 s for the live rule, against 0.46 s for Qwen on both lists.

A grid of $\tau_{\mathrm{u}} \in \{0.3, 0.4, 0.5, 0.6, 0.7\}$ and $\tau_{\mathrm{s}} \in \{0.60, 0.65, 0.70, 0.75, 0.80\}$ does not reverse the order. On UCF101 the live rule moves from 1332/3783 at (0.3, 0.60), with 0.94 uploads and a mean of 648 ms, to 1776/3783 at (0.7, 0.80), with 1.82 uploads and a mean of 913 ms. On HMDB51 the same corners are 487/1530 and 729/1530. The high corner is still short of the single Qwen pass. Raising both thresholds uploads more often and spends more compute to approach the model that already saw all three frames.

F. What these measurements do not support

The 60 images are a phrase-quality check. They do not measure how often the live gate uploads. The replay measures that count on three stored frames, not on a phone session and not on the Wi-Fi link. Neither measurement supports a claim about action start times. They do not compare schedulers. The class score is also a lower bound on descriptive usefulness: a phrase such as "Smiling" on a drinking image is the wrong class and may still be a fair description of the visible face. The report does not treat those misses as correct.

VII. CONCLUSION

The midterm system writes an activity phrase on the phone and uploads a picture only when that phrase is unsafe. A quiet frame, with a 32×32 grayscale mean absolute difference below 10, keeps the phrase and skips the phone model. A confident phrase must leave the ongoing action on two frames before the buffered frames are uploaded. A minimum token probability below 0.5 uploads on the current frame. The laptop then writes its own phrase and replaces the screen text only when the two phrases disagree. The displayed string is the model phrase.

On 60 still images the cosine matcher scores 38/60 for SmolVLM and 40/60 for Qwen, against 27/60 and 30/60 for character overlap. Routing on phone confidence reaches 41/60 to 43/60. On the three-frame replay the same live rule scores 1640/3783 on UCF101 and 660/1530 on HMDB51, below one Qwen pass at 1785/3783 and 812/1530, and above SmolVLM alone at 928/3783 and 288/1530. The quiet skip covers about a quarter to a third of the frames. The live rule still spends more compute than Qwen because a clip can call Qwen more than once after the phone model has already run. The replay does not include Wi-Fi delay, the phone's 4-bit runtime, or action boundaries.

The networking half is the work of 1–20 November. Checker traces under different JPEG sizes and laptop loads have to be collected, the predictor g_θ has to be trained on those traces, and round-robin, a static checking budget, and the predictive budget in (8) have to be compared in a multi-client simulation. A claim about action start times still needs labels for those boundaries. The three-frame replay does not provide them. Table VII records the shift. The original plan trained the predictor during 4–17 October and built the scheduler during 18–31 October. Those two blocks were spent on the sentence path and on the UCF101 and HMDB51 replay instead.

TABLE VII
SCHEDULE AND MIDTERM STATUS

Dates	Work	Status
23 Sep.–3 Oct.	Local inference and the Wi-Fi checking path	Done
4–17 Oct.	Open phrases, cosine check, and the 60-image sample	Done
18–31 Oct.	Quiet skip, two-frame leave, UCF101 and HMDB51 replay	Done
1–12 Nov.	Checker traces, predictor, uplink budget, scheduler	Planned
13–20 Nov.	Final evaluation, demo, report, and presentation	Planned

REFERENCES

[1] A. Marafioti, O. Zohar, M. Farré, M. Noyan, E. Bakouch, P. Cuenca, C. Zakka, L. B. Allal, A. Lozhkov, N. Tazi *et al.*, "Smolvlm: Redefining small and efficient multimodal models," *arXiv preprint arXiv:2504.05299*, 2025.

[2] L. S. S. Ray, B. Zhou, S. Suh, and P. Lukowicz, "Initial findings on sensor based open vocabulary activity recognition via text embedding inversion," *arXiv preprint arXiv:2501.07408*, 2025.

[3] H. Kang, Y. Park, Y. Yoo, Y. Choi, and S. J. Kim, "Open-ended hierarchical streaming video understanding with vision language models," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025.

[4] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, "Learning transferable visual models from natural language supervision," in *International conference on machine learning*. PmLR, 2021, pp. 8748–8763.

[5] P. Wang, S. Bai, S. Tan, S. Wang, Z. Fan, J. Bai *et al.*, "Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution," *arXiv preprint arXiv:2409.12191*, 2024.

[6] Y. Yao, T. Yu, A. Zhang, C. Wang, J. Cui, H. Zhu, T. Cai *et al.*, "MiniCPM-V: A GPT-4V level MLLM on your phone," *arXiv preprint arXiv:2408.01800*, 2024.

[7] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packed resources for general chinese embeddings," *arXiv preprint arXiv:2309.07597*, 2024.

[8] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," *ACM SIGARCH Computer Architecture News*, vol. 45, no. 1, pp. 615–629, 2017.

[9] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International conference on machine learning*. PMLR, 2023, pp. 19 274–19 286.

[10] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.

[11] Y. Jia, S. Tang, and Q. Yang, "Covspec: Efficient device-edge co-inference for vision-language models via speculative decoding," *arXiv preprint arXiv:2605.02218*, 2026.

[12] Y. Venkatesha, S. Kundu, and P. Panda, "Fast and cost-effective speculative edge-cloud decoding with early exits," *arXiv preprint arXiv:2505.21594*, 2025.

[13] C. Qian, X. Yu, Z. Huang, D. Li, Q. Ma, F. Dang, X. Ding, G. Shang, and Z. Yang, "edgevlm: Cloud-edge collaborative real-time vlm based on context transfer," *arXiv preprint arXiv:2508.12638*, 2025.

[14] N. Said and O. Landsiedel, "EdgeBoost: Confidence boosting for resource constrained inference via selective offloading," in *20th International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*. IEEE, 2024, pp. 11–18.

[15] A. Jalali, R. Padovani, and R. Pankaj, "Data throughput of CDMA-HDR a high efficiency-high data rate personal communication wireless system," in *IEEE 51st Vehicular Technology Conference Proceedings*, vol. 3, 2000, pp. 1854–1858.

[16] B. Briscoe, K. De Schepper, M. Bagnulo, and G. White, "Low latency, low loss, and scalable throughput (L4S) internet service: Architecture," RFC 9330, Tech. Rep., 2023.

[17] H. Xie, Z. Qin, G. Y. Li, and B.-H. Juang, "Deep learning enabled semantic communication systems," *IEEE Transactions on Signal Processing*, vol. 69, pp. 2663–2675, 2021.